

Trabajo práctico 2

Rolgar II

CB100 Algoritmos y Estructura de datos

Cátedra Schmidt

Integrantes:

- José Cabrera (112851) jacabrera@fi.uba.ar
- Mariano Argerich (112688) margerich@fi.uba.ar
- Emanuel Peña (108065) ejpena@fi.uba.ar
- Emanuel Lescano (113228) elescano@fi.uba.ar
- Santiago Guzzardi (113176) sguzzardi@fi.uba.ar
- Dylan Juli (114050) djuli@fi.uba.ar

Índice

Informe.....	3
Entorno de trabajo.....	3
Interpretación del enunciado.....	3
Problemáticas surgidas durante el desarrollo.....	5
Manual de programador.....	7
Estructura del proyecto.....	7
Paquetes.....	7
lib.....	8
test.....	8
app.....	8
útiles.....	8
bitmap.....	11
estructuras.....	13
modelo.....	14
rolgar2.....	19
configuraciones.....	31
elemento.....	32
carta.....	34
entidad.....	36
Manual de usuario.....	38
Descripción.....	38
Dificultades.....	38
Generación del mundo.....	39
Inventario.....	39
Enemigos.....	39
Cartas.....	39
Alianzas.....	40
Combate.....	40
Cuestionario.....	41
¿Qué es un SVN?.....	41
¿Qué es una “Ruta absoluta” o una “Ruta relativa”?.....	41
¿Qué es Git?.....	42

Informe

Entorno de trabajo

Se creó un repositorio en GitHub privado para el desarrollo del proyecto y un gráfico UML que se usó de manera orientativa para el diseño de clases.

En el repositorio se agregó un “proyecto” que contiene las tareas individuales que cada integrante puede elegir y páginas en la “wiki” en el que se redactaron por cada una, una descripción de un TDA en específico, su función, atributos y métodos.

También se utilizó la pestaña de Issues cada vez que se alguien encontró algún problema, bug, mejora, etc.

Cada integrante del grupo fue eligiendo tareas específicas, creando una branch y al finalizar creando una pull request. Se agregó en GitHub una protección a la rama principal para que no se pueda mergear directamente y que cada pull request requiera al menos de la aprobación de un integrante para poder ser mergeada a main.

Los nombres de las ramas y mensajes de los commits siguieron los siguientes estándares: [Conventional branch](#) y [Conventional commits](#).

Interpretación del enunciado

Al intentar interpretar algunas cosas del enunciado nos encontramos con las siguientes discrepancias entre los miembros del grupo y problemas al momento de modelar o desarrollar.

- **Tablero tridimensional:** se debatió la idea de hacer un juego que sea 3D, en el que la generación del mundo se vea afectada por la altura o que cada piso individual funcione independientemente de los demás. Se optó por la segunda opción ya que la generación del mapa se nos hizo más sencilla a

priori. Otra problemática que pensamos que podría existir fue la difícil interpretación que podría llegar a tener un jugador al visualizar el juego en 3D con una representación 2D por pisos (ya que elegimos esa forma de representar el juego).

Al principio dejamos como de las últimas tareas el desarrollo del tablero y el TDA celda (casillero del tablero), ya que para el punto inicial del proyecto aún no se había explicado en clases cuál sería su implementación con listas ni la representación visual con bitmaps.

- **Posición inicial de los personajes:** en el enunciado dice que la *posición inicial de un personaje* debe estar en el centro del tablero y luego se repite al momento de explicar los atributos del personaje. Imaginamos que hubo un error al crear el enunciado, pero aún así decidimos seguirlo lo mejor posible, así que siempre se crea un personaje en el centro del tablero.
- **Diseño de clases:** al momento de hacer las clases no se tuvo en cuenta la reusabilidad de cada clase en específico en algún otro juego sin relación con Rolgar II, por lo que muchas clases, métodos y atributos fueron cambiando mucho a lo largo del proyecto. Más aún con todo lo que fuimos viendo en clase y los problemas que nos fuimos encontrando durante el desarrollo.
- **Teclas de movimiento:** se decidió tomar la recomendación expresada en clase, las teclas por defecto para mover diagonalmente son Q (noroeste), E (noreste), Z (sudoeste) y C (sudeste).
- **Rampas:** sirven para mover a los jugadores entre pisos, pero se decidió que se haga automáticamente una vez el jugador ingresa a la rampa en vez de hacer que el jugador deba manualmente elegir la opción de subir o bajar la rampa.

Esto generó la incertidumbre de dónde debería moverse el jugador una vez ingresado en la rampa. No era una opción moverlo a la misma posición donde se encuentre la otra rampa en el siguiente nivel, ya que esto causaría

un bucle infinito. Por ese motivo las rampas ahora tienen orientación y dependiendo de esta la nueva posición del jugador es calculada.

- **Cartas:** las dos cartas que decidimos agregar al juego son
 - **Carta aleatoria:** genera una carta aleatoria al ser usada.
 - **Carta comodín:** genera una carta que puede elegir el jugador al ser usada.

Interpretamos que las cartas ataque doble y doble movimiento le permiten al jugador atacar y moverse dos veces respectivamente.

También que la carta de aumento de vida le permita al jugador que la usa recuperar vida, ya que existe la carta de curar a un aliado, pero no a uno mismo.

Problemáticas surgidas durante el desarrollo

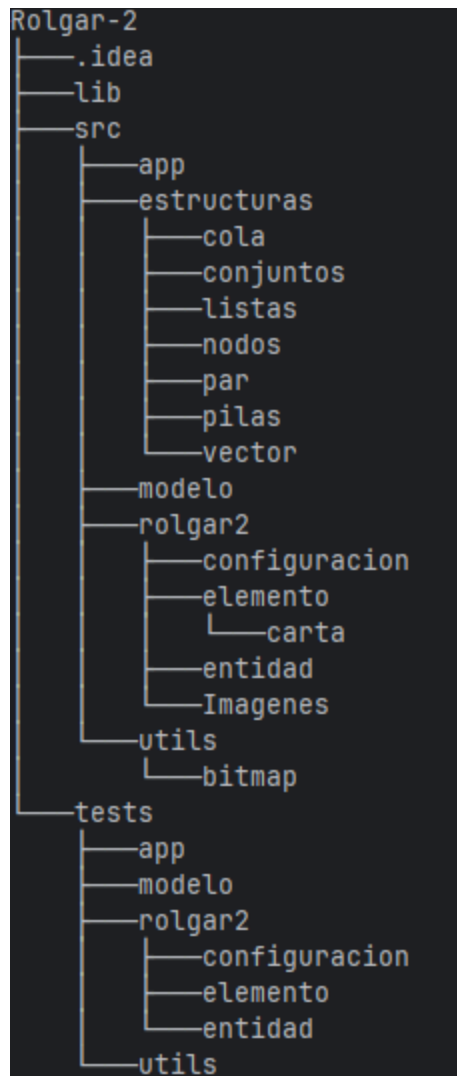
De la manera que se habían diseñado al inicio las clases había un árbol de dependencia en el cual no se podía avanzar cómodamente con el desarrollo de una clase sin tener otras previamente. Por tal motivo nos dimos cuenta que una posible solución para poder trabajar en paralelo hubiera sido, ya que teníamos diseñado el UML, hacer las firmas de las clases y que cada TDA sea independiente de los demás.

Por razones de escalabilidad decidimos hacer una estructura de jerarquía en la cual, por ejemplo, dos clases llamadas Personaje y Enemigo hereden de una superclase Entidad ya que evaluamos que las primeras podrían tener atributos en común. Para fechas cercanas a la entrega del trabajo entendimos que es más importante la reutilización de las clases.

Del material proporcionado en cada clase agregamos los tipos de datos, algunas validaciones, json, bitmap y Teclado. Además creamos un nuevo tipo de dato Par, que tiene la capacidad de almacenar dos tipos de datos distintos en una misma estructura. Esto fue necesario para poder guardar entidades y elementos en un mismo espacio de una manera ordenada.

Manual de programador

Estructura del proyecto



Paquetes

lib

Lib: Contiene librerías Java para trabajar con JSON.

- **jackson-annotations-2.17.2.jar**
 - **jackson-core-2.17.2.jar**
 - **jackson-databind-2.17.2.jar**
-

test

Test: El package test están las pruebas unitarias del trabajo, cubriendo la validación de su núcleo lógico (personajes, inventario y tablero) y herramientas del sistema, así como las mecánicas de combate, gestión de turnos y la carga de configuraciones y datos.

app

App: Establece el punto de entrada del programa. Se encarga de preparar el entorno inicializando el teclado y el sistema central Rolgar2, ejecuta el ciclo de juego y, una vez finalizado, cierra correctamente los recursos de entrada.

útiles

Útiles:

SistemaUtiles

Métodos

- void esperar(long milisegundo): detiene la ejecución del programa por el tiempo especificado. Valida que sea mayor que cero.
- String generarRutaAbsoluta(String rutaRelativa): Genera la ruta absoluta de un archivo del proyecto.

Teclado

Métodos

- void inicializar(): Inicializar el scanner para leer entrada del teclado.
- void finalizar(): Cierra el scanner y libera los recursos.
- String leerLinea(): Lee una línea completa ingresada por el usuario.
- int leerEntero(): Lee un número entero ingresado por el usuario. Sigue pidiendo hasta que se ingrese un entero válido.
- float leerFloat(): Lee un número float ingresado por el usuario. Sigue pidiendo hasta que se ingrese un float válido.
- char leerCaracter(): Lee el primer carácter ingresado por el usuario. Sigue pidiendo hasta que se ingrese al menos un carácter.

Temporizador

Atributos

- private static Instant tiempoSegmentoInicial.

- `private static Duration tiempoTranscurrido.`
- `private static boolean enMarcha.`

Métodos

- `static void pausar():` Pausa el temporizador y suma el tiempo transcurrido.
- `static void reiniciar():` Reinicia el temporizador y el tiempo guardado.
- `static boolean enMarcha():` Verifica si el temporizador está en marcha.
- `static Duration tiempoTranscurrido():` Obtiene el tiempo transcurrido sin formatear.
- `String getTiempoFormateado():` Obtiene el tiempo transcurrido formateado.

Validaciones

Métodos

- `static void validarMayorQueCero(double valor, String cadena):` Valida que el valor sea mayor a 0.
- `static void validarMayoroIgualCero(double valor, String cadena):` Valida que el valor sea mayor o igual a 0.
- `static void validarMayorIgual(double valor, double minimo, String cadena, String cadenaMinimo):` Valida que el valor sea mayor o igual al mínimo especificado.
- `static void validarDistintoDeNull(Object valor, String cadena):` Valida que el valor no sea null.

- static <T extends Number> void
validarRango(T valor, double desde, double hasta, String
nombreDelAtributo): Valida que el valor esté dentro del rango especificado
(inclusive).
 - static void validarCaracteresAlfabeticos(String valor, String
nombreDelAtributo): Valida que el texto contenga solo letras y espacios.
Acepta caracteres acentuados (á, é, í, ó, ú, ñ).
 - static void validarFalse(boolean valor, String nombreDelAtributo) : Valida que
el booleano sea false.
 - static void validarTrue(boolean valor, String nombreDelAtributo): Valida que
el booleano sea true.
-

bitmap

Atributos

- private int width
- private int height
- private BufferedImage image
- private double distanceToCamera

Métodos

- Bitmap(int width, int height): Genera un bitmap de ancho y alto dados.

- void drawPixel(int x, int y, Color color) : Dibuja un pixel de un color determinado.
- void drawLine(int x1, int y1, int x2, int y2, Color color) : Dibuja una línea del color dado.
- void drawRectangle(int x, int y, int width, int height, Color color): Dibuja un rectángulo del color dado.
- void drawCircle(int centerX, int centerY, int width, Color color): Dibuja un círculo del color dado.
- void drawText(String text, int x, int y, Font font, Color color, Color background): Dibuja un texto en la posición dada.
- void pasteBitmap(Bitmap other, int x, int y)
- String saveToFile(String path)
- static Bitmap loadFromFile(String path)
- int projectX(double x, double z)
- int projectY(double y, double z)
- void drawLine3D(double x1, double y1, double z1, double x2, double y2, double z2, Color color)
- void drawCube(double size, double cx, double cy, double cz, Color color)
- BufferedImage getImage(): Devuelve la imagen en memoria.
- int getWidth(): Devuelve el ancho.
- int getHeight(): Devuelve el alto.

BitmapViewer

Atributos

- private JFrame frame;
- private JPanel panel;
- private List<Bitmap> bitmaps;

Métodos

- BitmapViewer(List<Bitmap> bitmaps): Genera un visualizador de bitmaps con los bitmaps adjuntos.
- static void showBitmaps(Bitmap... bitmaps): Inicia el visualizador de bitmaps.
- void createAndShowGUI(): Genera una gráfica con los bitmaps. Luego los refresca cada 0.5 segundo
- void refreshImages(): Refresca las imágenes.

estructuras

Estructuras: Contiene la implementación de distintas estructuras de datos.

- listas
 - colas
 - pilas
 - vectores
 - nodos
 - pares
 - conjuntos
-

modelo

MODELO: Es una base en la que sus clases se pueden adaptar o heredar para construir la lógica de casi cualquier tipo de juego, desde un RPG complejo hasta un simple juego de puzzle o estrategia.

CLASE DADO

Generador de números aleatorios para mecánicas de juego

Atributos:

- caras: Rango de resultados posibles
- random: Generador estático compartido

Métodos:

- tirar(): Lanzamiento simple
 - getCaras(): Consulta de rango configurado
-

CLASE ELEMENTO

Base para todos los elementos del mapa

Atributos:

- Nombre: Crea un nuevo elemento con el nombre pasado por parámetro

Métodos:

- getNombre(): Devuelve el nombre del elemento

CLASE INVENTARIO

Contenedor de tamaño fijo para elementos

Atributos:

- datos: Array de elementos almacenados
- cantidad: Contador actual de elementos

Métodos:

- agregar(T): Añade elemento validando tipo y espacio
- obtener(int): Acceso por índice
- contiene(T): Búsqueda de elemento
- remover(int): Eliminación con reordenación
- estaLleno() / estaVacio(): Estados de capacidad
- getCapacidad() / getCantidad(): Métricas de uso

CLASE PERSONAJE

Base que representa al personaje

Atributos:

- Nombre: Crea un nuevo personaje con el nombre pasado
- Métodos:
- getNombre(): Devuelve el nombre del personaje
-

CLASE ALIANZA

Gestiona grupos cooperativos de personajes dentro del juego

Atributos:

- nombre: Identificador único de la alianza
- miembros: Colección de instancias de Personaje

Métodos:

- Constructor Alianza(String nombre): Crea alianza validando longitud y caracteres alfanuméricos
 - agregarMiembro(Personaje): Añade personaje validando capacidad máxima
 - quitarMiembro(Personaje): Elimina personaje existente
 - estaVacia(): true si no contiene miembros
 - estaLlena(): true si alcanzó límite configurado en Constantes
 - estaEnAlianza(Personaje): Verifica pertenencia de personaje
 - getNombre() / setNombre(String): Accesos para nombre con validación
 - getMiembros(): Retorna lista completa de miembros
 - getCantidadMiembros(): Retorna número actual de integrantes
-

CLASE TURNO

Gestor de secuencia de acciones por entidad

Atributos:

- personajes: Estructura round-robin de entidades
- indiceActual: indice que señala el turno

Métodos:

- siguienteTurno(): Avanza al siguiente en secuencia
 - removerPersonaje(personaje): Remueve el personaje de la secuencia
 - getPersonajeTurnoActual(): Se obtiene el personaje actual al que le toca el turno
 - getPersonajes(): Copia de la lista de personajes que componen la lista de turnos.
-

CLASE TABLERO

Contenedor principal del mapa 3D

Atributos:

- celdas: Estructura tridimensional de celdas
- Dimensiones: ancho, alto, profundo

Métodos:

- inicializarVecinos(): Establece relaciones de adyacencia
- obtenerVecinos(): Obtiene los vecinos de una celda en la posición

- `getCelda(int[])`: Acceso directo por coordenadas
 - `getVecinosDeCelda(int[])`: Retorna celdas circundantes
 - Métricos: `getCantidadDeCeldas()`, `getCantidadDeCeldasLibres()`
-

CLASE CELDA

Unidad fundamental del mapa tridimensional

Atributos:

- `contenidoCelda`: Elemento o entidad contenida
- `vecinos`: Matriz 3D de celdas adyacentes
- `posicion`: Vector de coordenadas $[x, y, z]$

Métodos:

- `estaLibre()`: true si no contiene elementos
 - `estaOcupada()`: true si contiene elemento/entidad
 - `getContenido()` / `setContenido()`: Accesores del contenido
 - `getVecinos()` / `setVecinos()`: Gestión de relaciones espaciales
 - `getPosicion()`: Retorna coordenadas de ubicación
-

PERSONAJE

Entidad controlada por jugador

Atributos adicionales:

- vision: Rango de percepción en celdas
- regeneracion: Porcentaje de curación por turno
- inventario: Contenedor de elementos
- escudo: Estado de protección activa

Métodos especializados:

- activarEscudo() / desactivarEscudo(): Gestión de defensas
- activarInvisibilidad() / desactivarInvisibilidad(): Control de visibilidad
- guardarElemento() / quitarElemento(): Gestión de inventario

rolgar2

ROLGAR2: Es la base controladora principal del juego y del bucle de ejecución central. Integra y gestiona todos los módulos esenciales (combate, alianzas, mapa) para asegurar un flujo de juego estructurado y coherente.

ENUM ACCIONES

Define todas las acciones que un jugador puede realizar durante su turno.

CLASE ADMINISTRADOR ACCIONES

Administra todas las acciones que pueden realizar los jugadores durante el juego.

Métodos:

- mover(Entidad): permite a una entidad moverse a una posición adyacente.
- mover(Entidad, int[]): mueve una entidad una posición específica.
- interactuar(Entidad): maneja la interacción de una entidad con los elementos en su celda.
- usarRampa(Entidad, Rampa): permite a una entidad usar una rampa.
- seleccionarCarta(Jugador, List<Jugador>): permite al jugador seleccionar y usar cartas de su inventario.
- crearAlianza(Jugador, Set<AlianzaRolgar2<Jugador>>): permite a un jugador crear una alianza.
- agregarJugadorAlianza(Jugador, AlianzaRolgar2<Jugador>): permite al líder agregar un miembro a la alianza.
- validarAlianzaJugador(Jugador, AlianzaRolgar2<Jugador>, List<Jugador>): verifica que el jugador no pertenezca a una alianza.
- quitarJugadorAlianza(Jugador, AlianzaRolgar2<Jugador> List<Jugador>): permite al líder de una alianza echar a un miembro.
- intercambiarCartas(Jugador, AlianzaRolgar2<Jugador>, List<Jugador>): permite intercambiar cartas entre 2 jugadores aliados.

- pausarJuego(): pausa el juego y muestra las instrucciones al jugador.
 - despausarJuego(): reanuda el juego después de una pausa.
 - abandonarJuego(Jugador, AlianzaRolgar2<Jugador>, List<Jugador>): elimina al jugador y su alianza si pertenece a una.
 - borrarAlianza(Jugador, AlianzaRolgar2<Jugador>, List<T>): elimina una alianza y todos sus miembros.
 - abandonarAlianza(Jugador, AlianzaRolgar2<Jugador>): permite a un jugador abandonar su alianza.
-

CLASE ALIANZA ROLGAR2

Representa una alianza en el juego.

Atributos:

- lider

Métodos:

- agregarMiembro(T): añade un miembro a la alianza.
 - quitarMiembro(T): elimina a un miembro de la alianza.
 - setLider(T): establece el líder de la alianza.
 - getLider(): obtiene el líder actual de la alianza.
-

CLASE COMBATE

Gestor de mecánicas de conflicto/ataques

entre entidades.

Atributos:

- minima_cantidad_perosnajes_combate, posicionCombate
- entidadesEnCombate

Métodos:

- escapar(Jugador): realiza un ataque de una entidad contra un objetivo en el combate.
- atacar(Entidad): verifica y filtra los objetivos válidos para atacar.
- verificarObjetivos(Entidad, List<Entidad>,): verifica si una entidad ha muerto
- verificarEntidadMuerta(Entidad): elimina del combate las entidades que se han movido fuera de la posición de combate.
- eliminarEntidadesFueraDeCombate(): elimina del coma
- calcularDanio(Entidad, Entidad): Calcula el daño infligido por un atacante a un objetivo.
- aplicarEscudo(): Aplica el efecto reductor del escudo al daño recibido.
- aplicarCritico(): Aplica el multiplicador de daño crítico al daño base.
- esCritico(): Determina si el ataque resulta en un golpe crítico.
- getEntidadesEnCombate(): obtiene la lista de entidades que están en combate.
- combateFinalizado(): verifica si el combate ha finalizado.

CLASE MAPA

Representa el mapa tridimensional del juego.

Atributos:

- random, tablero

Métodos:

- inicializarCeldas(): inicializa todas las celda del mapa con contenido vacío.
- generarMapa(List<Jugadores>, List<Enemigo>): genera el mundo completo.
- getEspacioMinimoRequerido(List<Jugador>, List<Enemigo>): calcula el espacio mínimo requerido para generar el mapa.
- esPosicionValidaPiedra(int[]): verifica si la posición es válida.
- getRampa(Celda<Par<List<T>>>, List<U>>>): obtiene la rampa de la posición.
- generarAnilloPiedras(): genera un anillo de piedras alrededor de cada piso del mapa.
- generarPiedras(): Generación de obstáculos perimetrales y aleatorios.
- generarAgua(): Distribución aleatoria de cuerpos de agua.
- generarCartas(): Colocación aleatoria de cartas coleccionables.
- generarRampas(): Creación de conectores entre niveles. (subida y bajada)
- getDireccionesValidasRampa(int[], int[]): obtiene las direcciones validar para colocar un par de rampas.

- `getVecino(int[], Direcciones)`: obtiene la celda vecina en una dirección específica.
- `getVecino(Celda<Par<List<T>>>, List<U>>>, Direcciones)`: obtiene la celda vecina en una dirección específica.
- `esVecinoValidoParaRampa(Celda<Par<List<T>, List<U>>>)`: verifica si un vecino es válido para colocar una rampa.
- `agregarPersonajes(List<T>)`: generación de personajes en el mapa.
- `agregarPersonaje(T)`: coloca un personaje en una posición aleatoria del mapa.
- `agregarPersonaje(T, int[])`: agrega un personaje en una posición específica del mapa.
- `quitarPersonaje(T, int[])`: quita un personaje de una posición específica del mapa.
- `agregarElemento(T, int[])`: agrega un elemento en X posición.
- `quitarElemento(U, int[])`: quita un elemento de X posición.
- `validarPosibleGeneracionRampas()`: valida que sea posible generar rampas.
- `contarPosicionesValidasRampas(nivel)`: cuenta cuantas posiciones válidas existen para colocar rampas.
- `getRampasMinimasRequeridas(nivel)`: obtiene cuantas rampas mínimas se requieren en un nivel específico.
- `validarEspacioDisponible()`: valida que haya espacio libre en el mapa.

- validarPosicion(int[]): debe ser distinto de null y cada coordenada mayor a cero.
- generarPosicionAleatoria(): genera una posición aleatoria dependiendo de las dimensiones del tablero.
- getCantidadEspacioVacio(): la cantidad de espacio vacío en el mapa.
- celdaVacía(int[]): si no contiene Personajes o Enemigos o bien vacía.
- tieneVecinoTraspasable(int[]): si tiene vecinos traspasables, al menos uno.
- esCeldaTraspasable(int[]): verifica si una celda es traspasable.
- hayPersonaje(int[]): verifica si hay al menos un personaje en la celda especificada.
- getDireccionContraria(Direcciones): obtiene la dirección contraria a la especificada.
- getEntidades(int[]): obtiene la lista de personajes en la posición especificada.
- getElementos(int[]): obtiene la lista de elementos en la posición especificada.
- getAncho(): obtiene el ancho del tablero.
- getAlto(): obtiene el alto del tablero.
- getProfundo(): obtiene la profundidad del tablero.

ENUM DIFICULTADES

Define los niveles de dificultad del juego.

ENUM DIRECCIONES

Define las direcciones de movimiento disponibles en el juego.

CLASE ENTRADA

Permite al usuario seleccionar un elemento de una lista.

Métodos:

- seleccionar(List<T>): permite al usuario seleccionar un elemento de una lista.
- ingresarDireccion(): solicita al usuario que ingrese una dirección válida.
- ingresarPosicionInventario(Jugador): solicita al usuario que ingrese una posición válida del inventario.
- ingresarAccion(List<Acciones>): solicita al usuario que ingrese una acción válida.
- ingresarDanio(): solicita al usuario que ingrese el daño de los enemigos.
- ingresesarCantidad(String): solicita al usuario que ingrese una cantidad positiva.
- ingresarJugadores(int): Crea los jugadores según la cantidad especificada.
- ingresarNombreAlianza(Set<AlianzaRolgar2<Jugador>>): solicita al usuario que ingrese un nombre único para una nueva alianza.
- obtenerTipoCarta(List<TipoCarta>): solicita al usuario que seleccione un tipo de carta valido.

- seleccionarDificultad(): permite al jugador seleccionar la dificultad del juego.
 - existeAlianza(String, Set<AlianzaRolgar2<Jugador>>): verifica si una alianza con el nombre especificado ya existe.
-

CLASE RENDER

Renderiza el tablero de juego. Genera bitmaps que representa la vista del jugador en el mapa 3D.

Atributos:

- tamaño_celda, bitmaps

Métodos:

- generarBitmapTablero(Jugador): genera los bitmaps del tablero desde la perspectiva del jugador.
 - dibujarBitmap(Bitmap, int[], int): dibuja un bitmap del tablero centrado en una posición específica.
 - mostrarCelda(int[], Bitmap, int, int): renderiza una celda específica en el bitmap.
 - mostrarBitmaps(Jugador): muestra los bitmaps generados en la interfaz gráfica.
-

CLASE SALIDA

Gestiona la interfaz de menús y mensajes del juego.

Métodos:

- `mostrarTitulo(String)`: muestra el título del juego en la consola con formato.
- `mostrarInstrucciones()`: muestra las instrucciones del juego.
- `mostrarDificultades()`: muestra las dificultades disponibles del juego con sus características.
- `mostrarMenu()`: muestra el menú principal del juego.
- `mostrarInterfazJugador(Jugador)`: muestra la interfaz del jugador con su información actual.
- `mostrarAccionesDelJugador(List<Acciones>)`: muestra las acciones disponibles que puede realizar un personaje en su turno.
- `mostrarSeparador()`: muestra un separador visual.
- `mostrarTiposDeCarta(List<TipoCarta>)`: muestra los tipos de carta disponibles para seleccionar.
- `mostrarListaDeJugadores(List<Jugador>)`: muestra una lista de jugadores numerada para seleccionar.
- `mostrarListaDePersonajes(List<? extends Personaje>)`: muestra una lista de personajes numerada para seleccionar.
- `mostrarInventario(Jugador)`: muestra el inventario de un jugador.

- `mostrarMensajeFinal(Set<AlianzaRolgar2<Jugador>>)`: muestra el mensaje final del juego con los ganadores.
 - `mostrarMensajeError(String)`: muestra un mensaje de error en la salida estándar.
 - `mostrarMensaje(String)`: muestra un mensaje en la salida estándar.
 - `mostrarMensajeMismaLinea(String)`: muestra un mensaje en la misma línea.
 - `esperar()`: pausa la ejecución del programa por un tiempo predeterminado.
-

CLASE TURNO ROLGAR2

Gestiona el sistema de turnos del juego.

Métodos:

- `siguienteTurno()`: avanza al siguiente turno, eliminando entidades muertas.
 - `avanzarTurnoHastaSiguienteEntidadViva()`: borra entidades hasta encontrar una viva.
 - `estaVivo(T)`: verifica si una entidad está viva.
 - `quickSortPorIniciativa(List<T>)`: ordena la lista de entidades por iniciativa usando QuickSort (de mayor a menor iniciativa)
 - `quickSortPorIniciativa(List<T>, int, int)`: maneja la partición e intercambio.
 - `particion(List<T>, int, int)`: divide e intercambia posiciones.
 - `intercambiar(List<T>, int, int)`: setea entidades entre posiciones.
-

CLASE ROLGAR2 (manejo de lógica principal)

Gestiona la inicialización, ejecución y control del flujo del juego. (Flujo)

Atributos:

- personajes, enemigos, mapa, combates

Métodos clave:

- `inicializar()`: inicializar el juego configurando todos los parámetros necesarios.
- `jugar()`: Bucle principal de juego. Manejo de turnos.
- `manejarAccionesJugador(Jugador)`: Manejo de acciones de usuario.
- `limpiarEntidadesMuertas()`: elimina del juego todas las entidades que han muerto.
- `verificarCombates()`: verifica el estado de los combates activos.
- `getAccionesDisponibles(Jugador)`: obtiene la lista de acciones disponibles para el jugador.
- `confirmarAccion(Acciones)`: solicita al jugador que confirme una acción.
- `juegoFinalizado()`: verifica si el juego ha terminado según las condiciones de victoria/derrota.
- `getAlianzas()`: obtiene todas las alianzas activas en el juego.
- `generarEnemigos(Dificultades, int)`: genera una lista de enemigos según la dificultad especificada.
- `getJugadores()`: obtiene una copia de la lista de jugadores activos.

- `getJugadoresSinAlianza()`: obtiene la lista de jugadores que no pertenecen a ninguna alianza.
 - `getMapa()`: obtiene el mapa del juego.
 - `agregarCombate(Combate)`: agrega un nuevo combate a la lista de combates activos.
 - `quitarCombate(Combate)`: remueve un combate de la lista de combates activos.
-

IMAGENES

Contiene todas las imágenes proyectadas para el bitmap.

configuraciones

CLASE CONFIGURACIONES

Gestiona parámetros personalizables vía JSON

- Controles: Mapping de teclas (WASD + diagonales)
- Símbolos: Caracteres visuales (Jugador: 'P', Enemigos: 'E', etc.)
- Persistencia: Archivo `config.json` para personalización

CLASE CONFIGURACIONES ROLGAR 2

Gestiona parámetros personalizables vía JSON para rolgar 2

- Controles de Rolgar 2: Mapping de teclas (WASD + diagonales)
- Símbolos de Rolgar 2: Caracteres visuales (Jugador: 'P', Enemigos: 'E', etc.)

- Constantes de Rolgar 2: Rangos
textuales,Dificultades,Generación,etc
- Persistencia de Rolgar 2: Archivo config.json para personalización

CLASE CONSTANTES

Centraliza todos los valores fijos del sistema

- Rangos textuales: Longitudes de nombres, descripciones
- Balance de juego: Vida, daño, probabilidades críticas
- Visual: Imágenes visuales (fondo,jugador,rampas,etc)
- Dificultades: Parámetros por modo (fácil, medio, difícil)
- Generación: Probabilidades de aparición de elementos

CLASE JSON

Utilidades para persistencia de configuración

- creacionDeJson(): Genera archivo con valores por defecto
- crearJsonPorDefecto(): Crea solo si no existe
- leerConfiguraciones(): Carga configuración existente

elemento

CLASE ELEMENTO ROLGAR 2

Base para todos los elementos del mapa de Rolgar2.

Atributos:

- Propiedades: lista de las propiedades específicas del juego

Métodos:

- `getPropiedades()`: Obtiene una copia de la lista de propiedades del elemento.
-

CLASE AGUA

Especialización de Elemento que modela cuerpos de agua con diferentes niveles de profundidad

Atributos:

- `nivel`: Entero que define la profundidad del cuerpo de agua

Métodos:

- Constructor `Agua(int nivel)`: Instancia un elemento agua con nivel específico
 - `getNivel()`: Retorna el valor numérico de profundidad actual
-

CLASE PIEDRA

Atributos:

- `traspasable`: false - Bloquea completamente el paso
-

CLASE RAMPA

Atributos:

- `traspasable`: true - Permite movimiento vertical

Métodos:

- `getDireccion`: Obtiene la dirección de la rampa.
 - `permiteSubir`: Booleano para movimiento ascendente/descendente
-

carta

CLASE CARTA

Identifica la carta

Métodos:

- Constructor `Carta(String nombre)`: Valida formato alfanumérico y longitud
 - `usarCarta(Personaje, int)`: Método abstracto para implementación específica
 - `getTipoDeCarta()`: Obtiene el tipo de carta
-

ENUM TIPO DE CARTA

Crea una instancia de la carta asociada a este tipo.

- Método: `usar(Jugador usuario, Jugador objetivo, TipoCarta tipo)`: Se utiliza en todos los tipos de cartas para su creación

CartaAtaqueDoble

- Efecto: Permite ejecutar dos ataques consecutivos en un turno
- Se valida alianza y ejecuta doble ataque

CartaComodín

- Efecto: Permite seleccionar cualquier carta disponible

- Se valida que exista el tipo de carta y la crea

CartaCuración

- Atributos: curacion - Puntos de vida a recuperar

CartaCuraciónAliado

- Atributos: curacion - Puntos de vida a transferir
- Valida alianza y cura objetivo

CartaDobleMovimiento

- Efecto: Permite dos desplazamientos en un turno

CartaEscudo

- Efecto: Activa protección contra daño
- Valida posesión y activa escudo

CartaInvisibilidad

- Efecto: Oculta al personaje de otros jugadores
- Valida posesión y activa invisibilidad

CartaRandom

- Efecto: Genera carta aleatoria del conjunto disponible
- Selección aleatoria basada en enum TipoCarta

CartaRoboCartaJugador

- Efecto: Transfiere carta aleatoria del inventario objetivo

CartaTeletransportación

- Efecto: Transloca personaje a posición aleatoria válida
-

entidad

CLASE ENTIDAD

Base para todos los actores del juego

Atributos:

- vida, vidaMaxima, combate
- fuerza, iniciativa, posición

Métodos:

- setVida(float): Establece la vida de la entidad.
 - setIniciativa(int): Establece iniciativa de la entidad.
 - setFuerza(float): Establece la fuerza de la entidad.
 - setPosicion(int[]): Establece posición de entidad en el mapa.
 - setCombate(Combate): Establece el combate con la entidad.
 - Getters de atributos y manejadores lógicos.
-

CLASE ENEMIGO

Entidad controlada por IA con atributos base de Entidad

CLASE JUGADOR

Representa a un jugador en el juego.

Atributos:

- vision, regeneración, inventario
- efectos, alianza

Métodos:

- guardarElemento(Carta): Guarda una carta en el inventario del jugador.
- quitarElemento(int): Elimina una carta del inventario de la posición indicada.
- agregarEfecto(Efectos): Agregar yub efecty de estado al jugador.
- eliminarEfecto(Efectos): Elimina un efecto de estado del jugador.
- regenerar(): Regenera vida del jugador según su tasa de regeneración.
- setVision(int): Establece el rango de vision del jugador.
- setRegeneracion(float): Establece la tasa de regeneración de vida del jugador.
- setAlianza(AlianzaRolgar2<Rolgar2>): Establece la alianza a la que pertenece el jugador.
- Métodos de comportamiento y getters de atributos.

ENUM EFECTOS

Define los efectos especiales que pueden aplicarse a los personajes.

Manual de usuario

Descripción

¿QUÉ ES ROLGAR 2?

Rolgar 2 es un juego de estrategia y supervivencia en un mundo tridimensional donde controlas personajes que exploran, combaten y forman alianzas en un mapa dividido en celdas.

Tu misión principal:

- Explorar el mapa tridimensional
- Enfrentar criaturas y otros jugadores
- Formar y gestionar alianzas
- Sobrevivir siendo el último en pie
- Utilizar cartas y elementos del entorno estratégicamente

Dificultades

1. Para Principiantes:
2. Forma alianzas temprano
3. Gestiona tu inventario cuidadosamente
4. Usa cartas defensivas cuando estés en desventaja
5. Explora sistemáticamente

Para Expertos:

1. Combina efectos de cartas para combos devastadores
2. Controla las rampas - ¡clave para el movimiento vertical!
3. Traiciona alianzas en el momento perfecto
4. Aprovecha la iniciativa - actuar primero es crucial

Consejos Esenciales:

1. Siempre verifica la alianza antes de atacar
2. Calcula tu fuerza vs. la vida del enemigo
3. Aprovecha el terreno y elementos del entorno

4. Los enemigos fuertes requieren trabajo en equipo
5. No ataques celdas vacías - ¡es un desperdicio!

Generación del mundo

Elementos del entorno: agua, piedras, rampas, etc.

Objetos y cartas para recolectar

Elementos del Entorno:

Elemento	¿Traspasable?	Efecto
Agua	Sí	Diferentes niveles de profundidad
Piedra	No	Crea laberintos y bloqueos
Rampa	Sí	¡Movimiento entre niveles!
Carta	Sí	Se añade automáticamente al inventario

Inventario

SISTEMA DE INVENTARIO

- Espacio limitado - ¡Gestiona sabiamente!
- Recolección automática al pisar objetos
- Sin espacio = No puedes recoger más objetos

Enemigos

- Criaturas hostiles que aparecen en el mapa:
 - NO pertenecen a alianzas
 - Comportamiento automático

Cartas

- Cartas Especiales - ¡Tu Ventaja Estratégica!
- Carta Efecto - Uso
- Ataque Doble - 2 ataques en 1 turno

- Curación -Recupera vida
- Doble Movimiento - 2 movimientos/turno
- Robo de Carta - Roba carta a enemigo
- Teletransportación - Mover a cualquier lugar
- Curación a Aliado - Cura compañeros
- Invisibilidad - Te vuelves invisible
- Escudo - Reduce daño recibido
- Carta Aleatoria - Obtienes carta random
- Carta Comodín - Elegir cualquier carta

Alianzas

PERSONAJES Y ALIANZAS

Atributos del Personaje:

- Vida: Tu resistencia ante los ataques
- Fuerza: Daño base que infliges
- Iniciativa: Determina quién actúa primero en cada turno
- Alianzas - ¡Juega en Equipo!
- Creación: Un jugador invita, otro acepta → ¡Alianza formada!

Beneficios:

- No puedes atacar a miembros de tu alianza
- Intercambia cartas entre aliados
- Combate cooperativo contra enemigos
- Nombre personalizado para tu equipo
- Expansión: Miembros existentes pueden invitar a nuevos jugadores

Combate

SISTEMA DE COMBATE

Cuando te encuentras en la misma celda que otro personaje se iniciará el combate si no pertenece a tu alianza. Una batalla por turnos donde podrás atacar, usar cartas o escapar definiendo tu estrategia en el momento.

Cálculo de Daño:

Tirada de dado (1-6) + Fuerza del atacante = Daño total

Ejemplo:

Jugador (Fuerza: 10) + Dado: 4 = 14 puntos de daño

DESPUÉS DEL COMBATE

- El objetivo recibe el daño calculado
- Nadie se mueve automáticamente
- Botín disponible si el enemigo es eliminado
- Decisión estratégica: ¿Retirarse, curarse, o seguir atacando?

En Rolgar 2, cada decisión importa. Desde la formación de alianzas hasta el uso estratégico de cartas, tu capacidad para adaptarte y planificar determinará si eres el último sobreviviente.

¡Prepárate para la batalla tridimensional definitiva!

Cuestionario

¿Qué es un SVN?

SVN es un sistema de control de versiones centralizado. Permite guardar y gestionar los cambios realizados en archivos (como código fuente o documentos) dentro de un repositorio central, para que varios usuarios puedan trabajar de forma coordinada y mantener un historial de versiones.

¿Qué es una “Ruta absoluta” o una “Ruta relativa”?

Ruta absoluta: indica la ubicación completa de un archivo o carpeta desde la raíz del sistema.

Ruta relativa: indica la ubicación de un archivo o carpeta con respecto al directorio actual.

¿Qué es Git?

Git es un sistema de control de versiones, que permite a varios desarrolladores trabajar en el mismo proyecto de forma simultánea. A diferencia de SVN, cada usuario tiene una copia completa del repositorio, lo que facilita el trabajo sin conexión y la fusión de cambios.